

Créer une application React Native sans Expo

Table des matières

I. Contexte	3
II. Les différences de création d'applications avec et sans Expo	3
III. Exercice : Appliquez la notion	7
IV. Génération de paquets applicatifs	7
V. Exercice : Appliquez la notion	13
VI. Essentiel	13
VII. Auto-évaluation	13
A. Exercice final.....	13

I. Contexte

Durée : 1 h

Environnement de travail : Un environnement de développement avec des émulateurs de terminaux

Pré-requis : Avoir fait le module sur l'utilisation des émulateurs

Contexte

Nous avons vu comment créer une application mobile en utilisant Expo et comment l'outil nous facilite grandement la tâche. Dans ce chapitre, nous allons voir la création d'une application sans utiliser Expo et ce que cela implique.

La plupart du temps, il restera cependant conseillé de démarrer un projet avec Expo et de voir si des limitations se présentent. Dans ce cas-là, on pourra migrer vers le workflow **Bare**, qui est très semblable à ce que nous allons voir dans ce module.

Si l'on souhaite avoir un contrôle total sur notre application ou bien ajouter React Native à une application Android/iOS déjà existante, alors créer une application React Native sans Expo peut être une solution intéressante.

Dans ce cours, nous verrons les différences entre ces modes, ainsi que des notions plus génériques, telles que la signature de paquets applicatifs et le fonctionnement de l'outil en ligne de commande de React Native.

II. Les différences de création d'applications avec et sans Expo

Objectifs

- Comprendre les principales différences entre les deux manières de fonctionner
- S'initier à la création d'applications sans Expo

Mise en situation

Dans ce module, nous allons voir les éléments de base liés à la création d'une application React Native autonome qui n'utilise pas Expo. Pour cela, nous allons mettre en place un nouveau projet sans Expo, puis nous étudierons la notion d'*auto-linking* avant de voir comment signer les paquets applicatifs.

À noter que la plus grosse différence se situe dans le fait que l'on est obligé de compiler nous-mêmes, et donc d'avoir un environnement de compilation spécifique à la plateforme cible. Il y a également un peu moins d'outils de développement lorsque l'on n'utilise pas Expo.

Rappel Attention

Lorsque l'on n'utilise pas Expo, il faut posséder un Mac pour pouvoir développer sur iOS.

Créer un nouveau projet

La première chose à faire est donc d'utiliser la CLI : notons bien le mot « utiliser » et non pas « installer ». En effet, auparavant, il était recommandé de l'installer, mais pour plus de flexibilité, il est désormais recommandé d'utiliser toujours la dernière version en utilisant `npx`. Nous avons vu précédemment dans ce cours que cet outil permet d'exécuter un programme NodeJS.

Mais avant, vérifions que nous avons bien NodeJS et npm d'installés en tapant `node -v` et `npm -v`. Si ce n'est pas le cas ou que la version de NodeJS est antérieure à la version 10, il faut récupérer une version plus récente ici : <https://nodejs.org/fr/download/>.

Lançons ensuite la commande `npx @react-native-community/cli init <nom_du_projet_sans_espaces>` dans un dossier spécialement créé pour nos projets.

Conseil

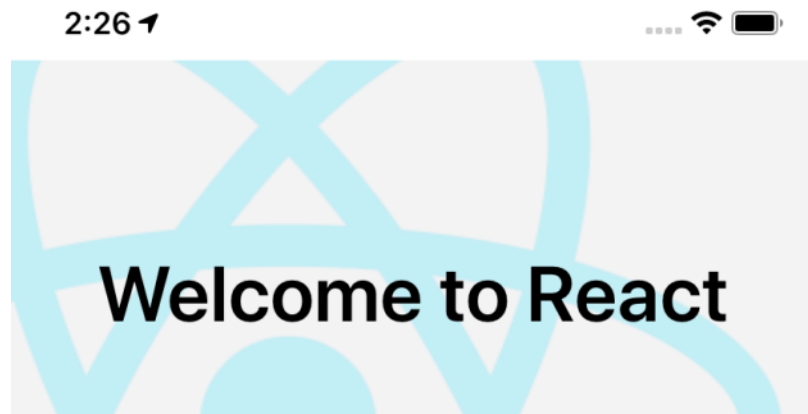
En cas de soucis, il ne faut pas hésiter à lancer la commande `npx react-native doctor` qui donnera plus d'informations sur d'éventuels problèmes qui empêcheraient d'exécuter la procédure décrite en amont.

Lancement de l'application

Une fois l'initialisation du projet terminée, il faut se rendre dans le dossier du projet et lancer les commandes `npx react-native run-ios` ou `npx react-native run-android` selon si l'on souhaite utiliser l'émulateur iOS, Android ou bien lancer l'application sur un téléphone branché.

En lançant cette commande, Xcode (l'IDE pour développer des applications iOS) ou Gradle (le gestionnaire de build d'Android) va lancer un build de l'application en mode développement et va automatiquement installer cette dernière sur le terminal ou sur l'émulateur. Ici, il n'y a pas d'application universelle pour tous les projets (comme Expo le permet) et chaque projet possède sa propre application dès le développement.

Nous devrions donc arriver sur cet écran et, ici, pas grand-chose ne change par rapport à Expo, hormis que nous n'avons plus accès à tous les outils que nous avons vus précédemment. Il faudra également installer les dépendances non incluses dans le cœur de React Native à la main, en suivant à chaque fois une procédure d'installation spécifique.



Step One

Edit **App.js** to change this screen and then come back to see your edits.

See Your Changes

Press **Cmd + R** in the simulator to reload your app's code.

Debug

Press **Cmd + D** in the simulator or **Shake** your device to open the React Native debug menu.

Learn More

Read the docs to discover what to do next:

[The Basics](#)

Explains a Hello World for React Native.

[Style](#)

Covers how to use the non named style which

Le concept d'auto-linking

Lorsqu'on souhaite installer une nouvelle librairie, React Native possède un mécanisme très pratique qui est très souvent disponible (c'est à la charge du créateur de la librairie de l'implémenter) : l'*auto-linking*.

Depuis la version 0.60, React Native a introduit un procédé d'auto-linking qui permet d'automatiser en grande partie l'installation des dépendances natives externes. La majorité des librairies se conforme à cette technique et l'installation est donc transparente (mais ce n'est malheureusement pas le cas pour toutes les dépendances).

Si l'auto-linking est disponible, il suffira d'installer le paquet applicatif via npm et de re-build l'application pour que le code natif de la librairie tierce soit inclus et compilé (désormais, à chaque ajout de code natif, il faut recompiler en relançant la commande de développement vue plus haut grâce à la CLI).

Ainsi, pour installer un module natif, il faudra suivre une procédure spécifique, détaillée dans la documentation de la librairie. Celle-ci peut grandement varier d'une librairie à une autre, car l'écosystème est ouvert et que chacun a ses propres pratiques.

Cocoapods (Mac seulement)

Sur Mac, il est nécessaire d'utiliser un gestionnaire de dépendances spécifique : **Cocoapods**. Celui-ci fournit une commande `pod install` qui permet d'installer les paquets applicatifs natifs d'iOS. C'est en quelque sorte le npm pour iOS et l'Objective-C.

Pour cela, il faut se rendre dans le dossier `ios` du projet et taper la commande `pod install` à chaque ajout de librairie native, afin que les dépendances natives iOS soient également téléchargées avant d'être compilées.

Si la commande échoue, c'est peut-être parce que **Cocoapods** n'est pas installé. Dans ce cas, il faudra lancer `sudo gem install cocoapods`.

Complément « On doit tout faire soi-même »

Maintenant qu'on a vu les bases, il faut savoir qu'on peut coder son propre code natif dans le dossier `android` ou le dossier `ios`. Il faudra cependant bien penser à re-compiler le code à chaque fois.

Cette partie concernant le développement natif, elle ne sera pas détaillée dans ce cours. Cependant, il peut être intéressant de se renseigner sur Java Android ou Kotlin Android et/ou Objective-C ou Swift, pour une utilisation ultérieure ou pour aller plus loin.

Remarque Signature de paquets applicatifs

Il nous incombe également de signer nous-mêmes les paquets applicatifs `.ipa` et `.apk` pour iOS et Android respectivement, avant l'envoi sur les stores.

Ce n'est pas une manipulation nécessaire en mode développement et nous ne nous y attarderons pas dans ce cours.

Syntaxe À retenir

- Bien qu'il soit généralement plus intéressant de commencer avec un projet Expo qui migrera vers un Bare Workflow, il y aura certains cas où l'on aura besoin de réaliser des applications React Native en dehors d'Expo.
- Cela a certains pré-requis, comme le fait de devoir compiler soi-même les paquets applicatifs, les signer et surtout mettre en place un environnement de compilation pour chaque plateforme, mais également de traiter soi-même l'ajout des dépendances externes qui utilisent du code natif.
- React Native met pour cela à disposition un mode appelé auto-linking, présent dans la plupart des librairies externes populaires.

Complément

- <https://github.com/react-native-community/cli/blob/master/docs/autolinking.md>

Exercice : Appliquez la notion

Exercice

Les paquets applicatifs Android ont une extension de fichier en...

- ☐ .aab
- ☐ .ipa
- ☐ .apk

Exercice

À quoi sert la fonctionnalité d'auto-linking de React Native ?

- ☐ Elle sert à installer automatiquement les paquets sans avoir à utiliser npm : il suffit d'importer les paquets comme sur la version web d'Expo
- ☐ Elle sert à automatiquement lier les dépendances natives des paquets annexes que l'on installe pour qu'ils compilent avec notre application
- ☐ Aucune des deux réponses précédentes

Exercice

L'ajout d'une librairie pour iOS dans un projet sans Expo...

- ☐ Nécessite l'utilisation d'une commande spécifique : `pod install`
- ☐ Peut se faire automatiquement si l'auto-linking a été configuré par ses créateurs

IV. Génération de paquets applicatifs**Objectifs**

- Comprendre le processus de compilation d'une application React Native
- Voir les différents types d'archives et les configurations liées à la compilation d'une application

Mise en situation

Dans ce chapitre, nous allons voir les méthodes pour générer des archives applicatives sans utiliser Expo. Ces tâches sont, dans le cadre d'une application Expo, réalisées de manière invisible par l'outil, mais, ici, elles devront être traitées manuellement.

Attention

Pour réaliser ces opérations pour iOS, il faudra posséder un Macbook.

Générer une archive Android AAB ou un APK

Pour livrer une application sur un terminal Android ou sur le magasin d'applications Google Play, il faut générer une archive Android (AAB) ou un paquet applicatif Android (APK).

Pour cela, nous allons utiliser Gradle, qui est un utilitaire permettant de définir une configuration et une liste de tâches à exécuter. Sur Android, la configuration qui nous intéresse se situe dans le fichier `android/app/build.gradle` : il va falloir la remplir pour mener à bien l'opération.

```

1 defaultConfig {
2     applicationId "com.monapp" // Ici c'est le nom du package de notre application
3     minSdkVersion rootProject.ext.minSdkVersion // La version minimum d'Android requise
4     targetSdkVersion rootProject.ext.targetSdkVersion // La version cible
5     versionCode 1 // Le versionCode à incrémenter à chaque fois
6     versionName "1.0" // La version affichée à nos utilisateurs et dans le store
7 }
8 // ...
9 signingConfigs {
10     debug { // Pour notre environnement de developpement, sur android nommé debug
11         // Ici on utilise un keystore de debug, avec les infos liées
12         storeFile file('debug.keystore')
13         storePassword 'android'
14         keyAlias 'androiddebugkey'
15         keyPassword 'android'
16     }
17 }
18 buildTypes {
19     // Ici est déclaré notre build de développement nommé debug
20     debug {
21         // Il utilise la configuration de signature vue plus haut
22         signingConfig signingConfigs.debug
23     }
24     release {
25         // Caution! In production, you need to generate your own keystore file.
26         // see https://reactnative.dev/docs/signed-apk-android.
27
28         // Ici pour tester on utilise la configuration de debug également, mais comme le
29         // montre le warning plus haut
30         // il faut générer un keystore de production
31         signingConfig signingConfigs.debug
32
33         // Ici ce sont des optimisations pour la production
34         minifyEnabled enableProguardInReleaseBuilds
35         proguardFiles getDefaultProguardFile("proguard-android.txt"), "proguard-rules.pro"
36     }
37 }

```

Une fois cette configuration réalisée, il faut se rendre à la racine du dossier `android` dans le projet et lancer la commande suivante : `./gradlew bundleRelease`.

Cela va générer une archive AAB, qui est une archive non exploitable telle quelle par les terminaux Android. Par contre, Google Play, en la récupérant, pourra l'utiliser pour générer un APK par type de terminal Android.

Complément **Keychain de production**

La signature des paquets nécessite la mise en place de certificats signés et encodés en RSA grâce à une clé publique et une clé privée. On appelle l'ensemble de ces éléments la **keychain**.

On remarque ici que les éléments utilisés pour signer nos paquets sont ceux de test :

```
1 signingConfigs {
2     debug { // Pour notre environnement de developpement, sur android nommé debug
3         // Ici on utilise un keystore de debug, avec les infos liées
4         storeFile file('debug.keystore')
5         storePassword 'android'
6         keyAlias 'androiddebugkey'
7         keyPassword 'android'
8     }
9 }
```

Afin de générer une **keychain** de production, on pourra utiliser un outil proposé par Oracle nommé **keytool**, qui se trouve généralement dans `C:\Program Files\Java\jdkx.x.x_x\bin` sous Windows.

Cet outil s'utilise en ligne de commande et prend en paramètres :

- **genkeypair** : demande la génération d'une paire de clés, publique et privée
- **keystore** : crée un **keystore**, qui est un fichier de stockage crypté contenant des clés et des certificats
- **alias** : spécifie l'alias qui sera utilisé pour identifier une clé ou un certificat dans le **keystore**
- **keyalg** : définit l'algorithme d'encodage utilisé
- **keysize** : définit la longueur de la clé de chiffrement utilisée
- **validity** : détermine la durée de validité du certificat

La commande utilisée est donc sous la forme suivante :

```
1 keytool -genkeypair -v -keystore my-upload-key.keystore -alias my-key-alias -keyalg RSA -
   keysize 2048 -validity 10000
```

Il faudra ensuite placer le fichier généré dans `android/app` et éditer un fichier `~/.gradle/gradle.properties` (plus sécurisé, car dans le dossier `home`, et donc pas versionné) ou `android/gradle.properties` et remplacer les ***** par les bonnes valeurs de notre projet.

```
1 MYAPP_UPLOAD_STORE_FILE=my-upload-key.keystore
2 MYAPP_UPLOAD_KEY_ALIAS=my-key-alias
3 MYAPP_UPLOAD_STORE_PASSWORD=*****
4 MYAPP_UPLOAD_KEY_PASSWORD=*****
```

Il suffit ensuite d'éditer `android/app/build.gradle` comme suit :

```
1 android {
2     ...
3     defaultConfig { ... }
4     signingConfigs {
5         release {
6             if (project.hasProperty('MYAPP_UPLOAD_STORE_FILE')) {
7                 storeFile file(MYAPP_UPLOAD_STORE_FILE)
8                 storePassword MYAPP_UPLOAD_STORE_PASSWORD
9                 keyAlias MYAPP_UPLOAD_KEY_ALIAS
10                keyPassword MYAPP_UPLOAD_KEY_PASSWORD
11            }
12        }
13    }
14    buildTypes {
15        release {
16            ...
17            signingConfig signingConfigs.release
18        }
19    }
20 }
```

```
18     }
19   }
20 }
```

Remarque

Contrairement à Apple qui possède le *software* et le *hardware*, sur Android, il existe une multitude de fabricants de téléphones qui n'utilisent pas les mêmes composants physiques. Il faut donc adapter le build au terminal cible, mais cela ne nous concerne pas directement en utilisant un AAB. Cela dit, Apple procède aussi à des optimisations en fonction du terminal cible.

Si l'on souhaite générer un APK universel (qui est assez gros, car il contient toutes les librairies, même celles qui ne sont pas utilisées, mais qui fonctionne sur tous les téléphones), on pourra lancer la commande : `./gradlew assembleRelease`.

Ces deux commandes vont enclencher un processus de compilation et, peu importe la commande lancée, les fichiers seront générés dans le dossier `android/app/build/outputs`.

Il suffira alors de les télécharger sur Google Play comme décrit ici : <https://support.google.com/googleplay/android-developer/answer/113469?hl=fr>.

Complément Tester sans générer un package

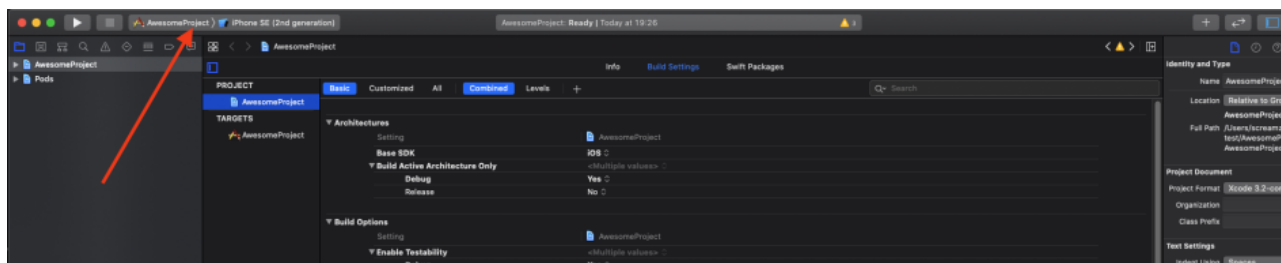
Pour tester le build directement sur l'émulateur ou le téléphone, on utilisera la commande : `npx react-native run-android --variant=release`.

Générer un IPA pour iOS

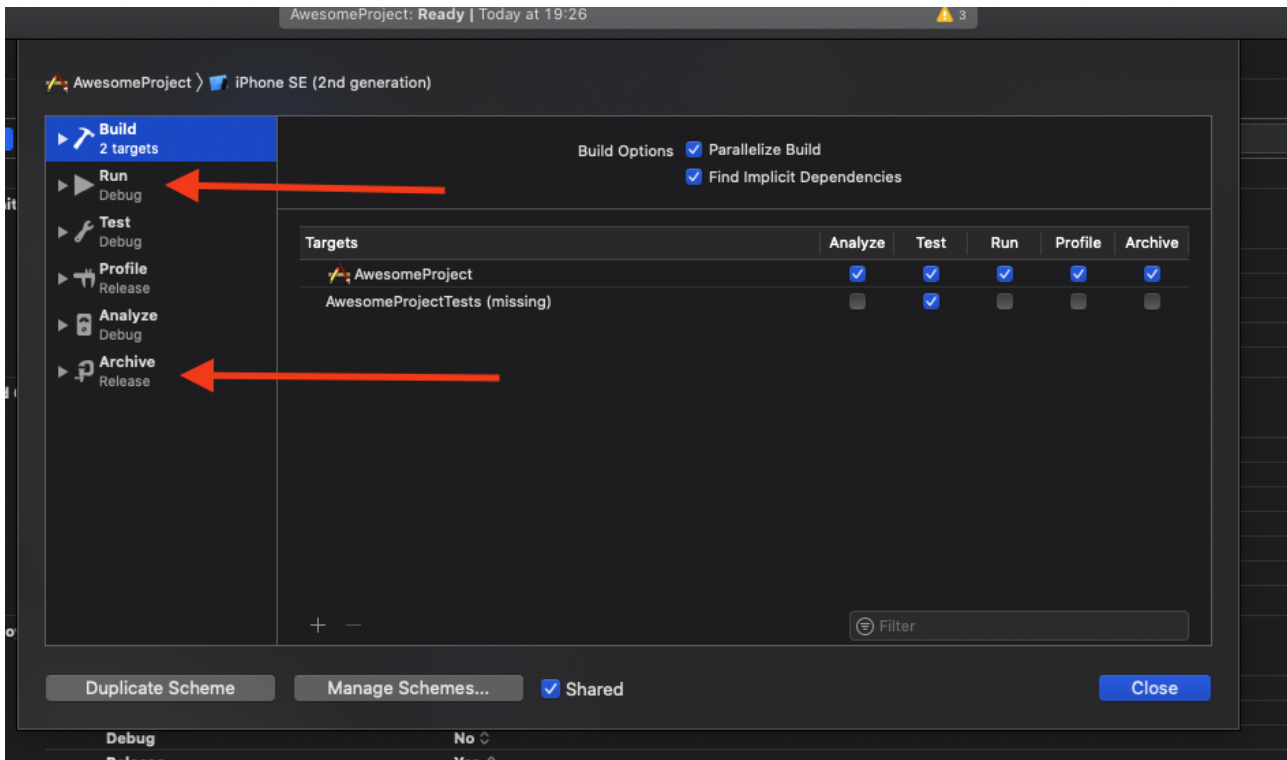
Pour la compilation sur iOS, c'est assez différent, et les premières notions à comprendre sont celles de « schémas » (*schemes*) et de « cibles » (*targets*) dans XCode :

- Les targets correspondent aux packages binaires qui seront générés pour notre archive. Grossièrement, 1 target = 1 binaire = 1 application. La plupart du temps, nous n'aurons qu'une seule target, mais on pourrait par exemple développer une extension du centre de notifications qui requiert un binaire autonome à installer sur les terminaux pour fonctionner.
- Un scheme est un *preset* de configurations.

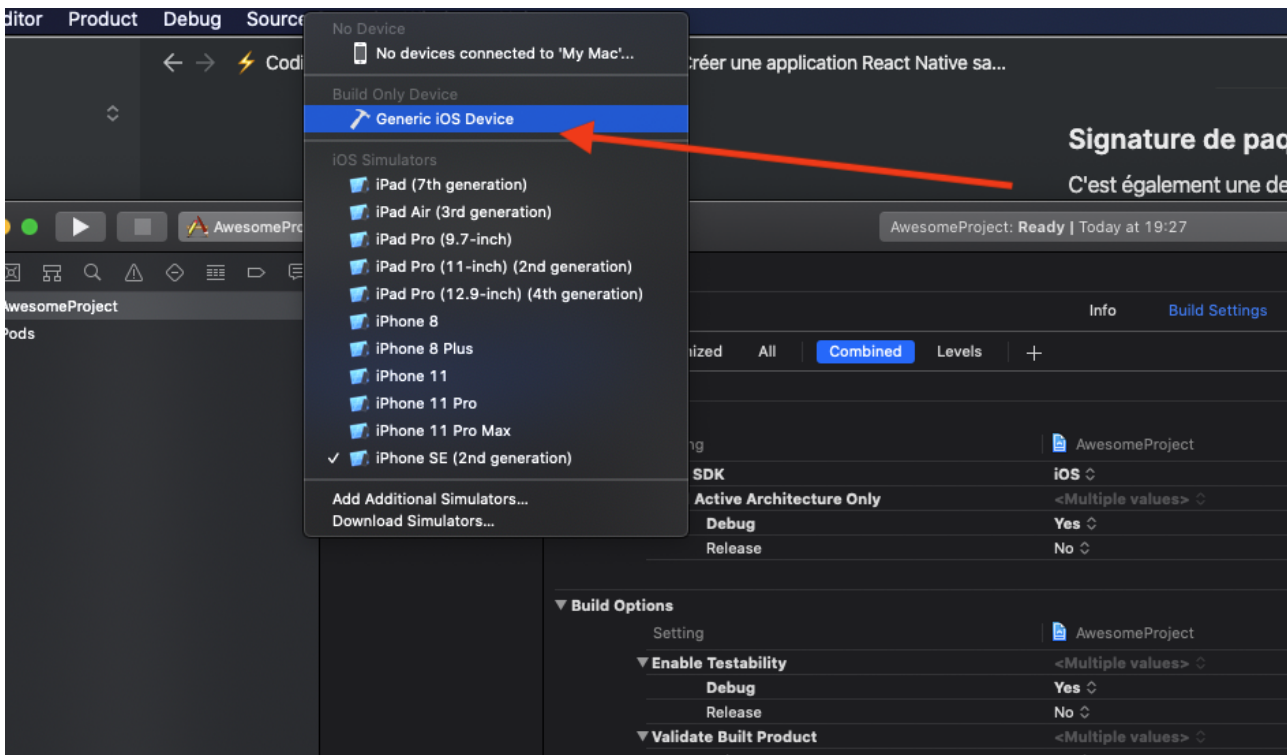
La génération d'un IPA requiert l'utilisation de schemes qui peuvent avoir plusieurs targets. Ainsi, dans l'exemple ci-dessous, nous n'avons qu'un seul scheme qui porte le nom de notre application, « *Awesome Project* » :



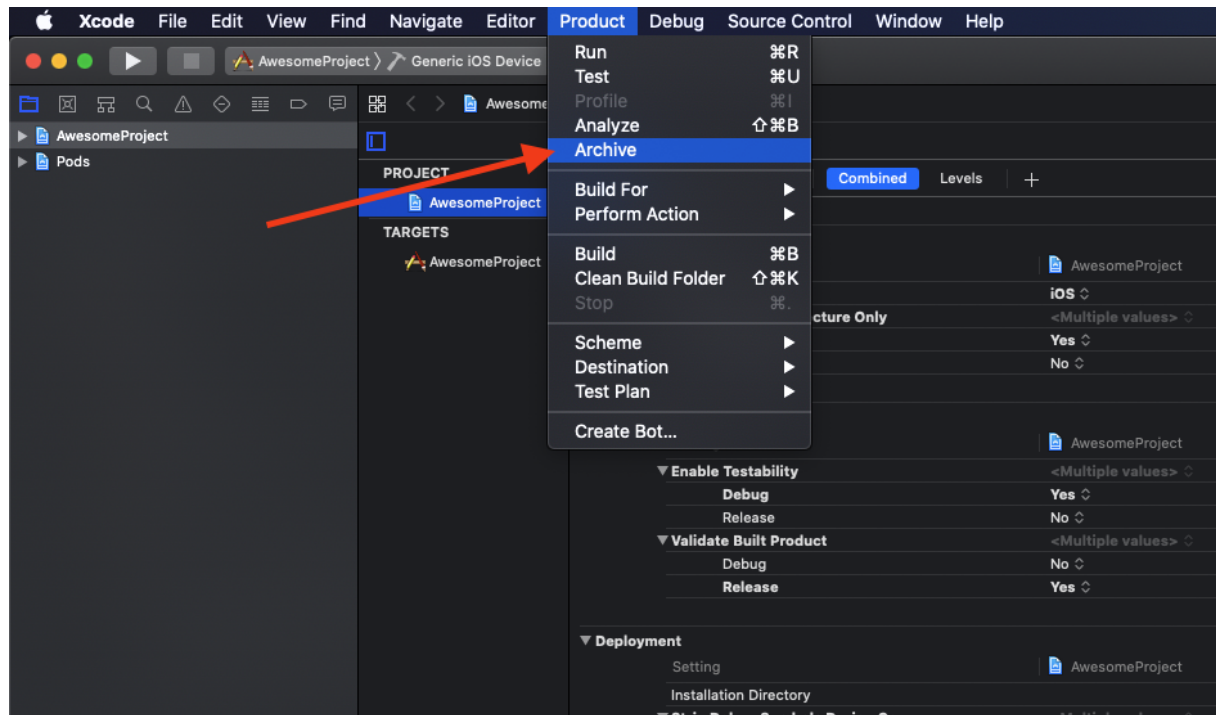
Si l'on va dans le détail de ce scheme, on voit que la tâche Run utilise la configuration de développement et que la tâche Archive (le procédé de build pour envoyer sur l'AppStore) utilise la configuration release, c'est-à-dire optimisée pour la production.



À droite du schéma, on peut choisir le type d'émulateur à utiliser pour l'action que l'on souhaite effectuer. Pour builder, il faudra choisir l'option *Generic iOS device*, car on cible tous les types de terminaux :

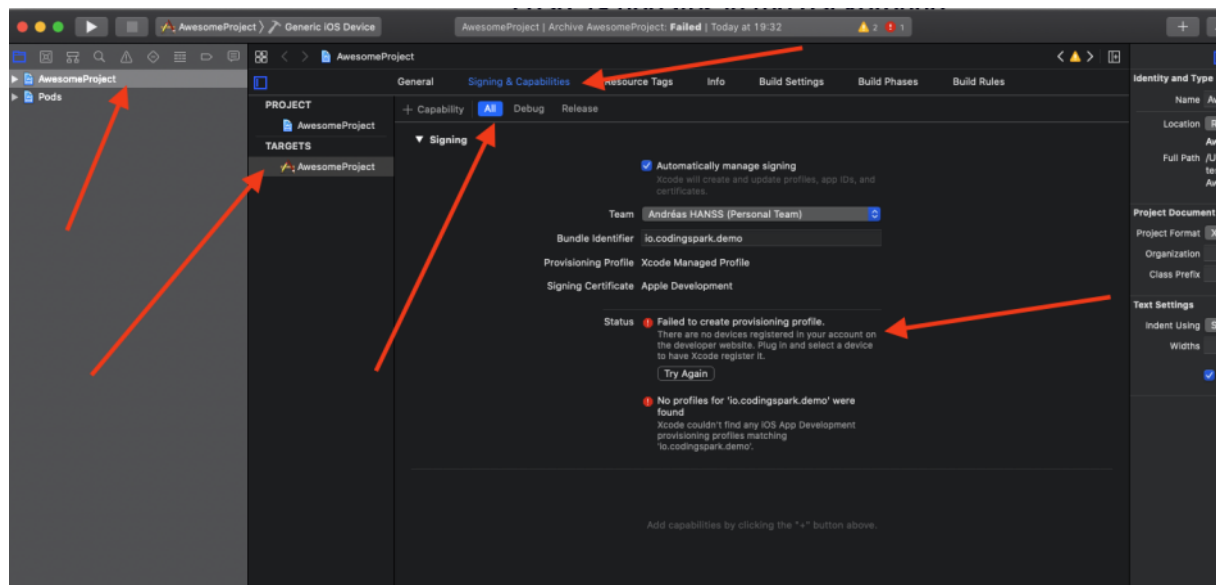


On va ensuite dans le menu *Product*, puis *Archive*, pour lancer la compilation.



Ici, XCode commence à builder notre package : on peut observer l'avancement et les erreurs dans la barre en haut de l'interface.

Cette étape nécessite d'avoir un compte Apple Developer configuré. Comme on peut le voir dans la capture d'écran ci-dessous, la compilation échoue si ça n'est pas le cas :



Nous ne pourrions pas aller plus loin dans le cadre de ce cours, car Apple est très restrictif sur son environnement et verrouille beaucoup de choses sans compte de développement. Mais la suite est très simple et nous affiche une fenêtre avec notre archive et un bouton **Envoyer sur l'AppStore** qui guide le développeur dans le processus de signature et de téléchargement.

Il faudra ensuite se rendre sur AppStore Connect, sur <http://developer.apple.com>, afin de configurer la fiche technique de l'application et soumettre le build à l'essai pour la review technique par les équipes d'Apple.

De là, nous pourrions faire un `TestFlight`, qui est une fonctionnalité de tests fermés d'Apple pour distribuer l'application jusqu'à 2000 utilisateurs pendant une période limitée, ou bien valider la publication sur le vrai AppStore pour que tout le monde puisse la télécharger.

Syntaxe À retenir

- Le build Android est géré par Gradle et permet de générer une archive Android (AAB) qui est optimisée pour Google Play, ou un APK universel que l'on va pouvoir installer sur n'importe quel téléphone.
- Le build iOS, lui, se déroule entièrement dans XCode et définit plusieurs notions comme les schemes ou les targets, qui permettent de le structurer.
- Dans tous les cas, la configuration reste à notre charge, ainsi que la résolution des problèmes de compilation de code natif.

Complément

- <https://reactnative.dev/docs/signed-apk-android>

V. Exercice : Appliquez la notion

Question

Dans cet exercice, il faut imaginer à quoi ressemblerait le fichier de configuration de `android/app/build.gradle` pour une application Android nommée **supercalcullette**, créée par la société Aurora. L'application est en version 2.0.5.

VI. Essentiel

La création d'une application React Native sans Expo est très semblable pour la partie JavaScript. Cependant, on se rend compte que, pour la partie devops/compilation, nous sommes beaucoup plus libres sans utiliser Expo.

Être plus libre signifie également qu'il y a moins de choses magiques et que l'on devra configurer et exécuter plus de tâches nous-mêmes. Pour cela, il faudra se référer majoritairement à la documentation Android ou iOS, car ces notions concernent moins directement React Native en lui-même.

VII. Auto-évaluation

A. Exercice final

Exercice 1

Exercice

Il est possible d'utiliser des composants Expo dans React Native *standalone*.

- ☐ Vrai
- ☐ Faux

Exercice

Pour générer un paquet applicatif prêt pour la production sur Android, j'utilise...

- ☐ ./gradlew bundleRelease
- ☐ ./gradlew generateRelease
- ☐ ./gradle release

Exercice

Pour générer un paquet applicatif prêt pour la production sur iOS, j'utilise...

- ☐ npx react-native release-ios
- ☐ Xcode et je fais product → Archive

Exercice

À quoi sert la clé minSdkVersion dans la configuration de Gradle ?

- ☐ On y définit le niveau minimum de l'API pour React Native
- ☐ On y définit le niveau minimum de la version d'Android

Exercice

Quelle est la version minimum de NodeJS requise par React Native ?

- ☐ 8
- ☐ 9
- ☐ 10
- ☐ 12

Exercice

Dans quel fichier définit-on l'identifiant de l'application sur Android ?

- ☐ android/app/build.gradle
- ☐ android/build.gradle
- ☐ android/settings.gradle

Exercice

Pour lancer la génération d'une archive . IPA pour iOS, je vais dans le menu...

- ☐ Build → Generate IPA
- ☐ Product → Archive
- ☐ File → Build Application

Exercice

Il est nécessaire d'avoir un compte Apple Developer pour générer une IPA.

- ☐ Vrai
- ☐ Faux

Exercice

Dans la configuration de Gradle pour Android, quelle est la différence entre le `versionCode` et le `versionName` ?

- ☐ Aucun, c'est juste un alias pour la même chose
- ☐ Le `versionCode` est un nombre qui doit être incrémenté à chaque nouveau build, alors que le `versionName`, non
- ☐ Le `versionCode` n'apparaît pas à l'utilisateur sur Google Play, alors que le `versionName`, oui

Exercice

Comment s'appelle le gestionnaire de paquets spécifiques à iOS vu dans ce chapitre ?

- ☐ cocoapods
- ☐ cocopod
- ☐ npm
- ☐ iospm

